



Real-time AI Sign Language Translator

■ Final Year Project ■ AI / Deep Learning 40 Signs Supported
95%+ Accuracy ■ Text-to-Speech ■ Sentence Builder

BTech CSE (AI & ML) | Final Year

Python 3.11	FastAPI	TensorFlow	MediaPipe	React 18	gTTS
-------------	---------	------------	-----------	----------	------

■ Project Overview

SignAI is a real-time sign language translator that uses a webcam to detect hand signs and converts them into text and speech. It is built with a React frontend, FastAPI backend, MediaPipe for hand landmark detection, and a stacked LSTM neural network for classification.

Step	Component	Description
1	Webcam	Captures live video at 15 fps
2	MediaPipe Hands	Extracts 21 hand landmarks (63 values)
3	Sliding Window	Groups 30 frames into one sequence
4	LSTM Model	Predicts sign with confidence score
5	FastAPI WebSocket	Streams prediction to React frontend
6	Sentence Builder	Builds words into full sentences
7	gTTS / Browser TTS	Speaks the sentence aloud

Category	Signs
Common Phrases	hello, yes, no, thankyou, please, sorry, help, good, bad, stop
Expressions	iloveyou, goodmorning, goodnight, howareyou, fine
Actions	eat, drink, sleep, come, go
Alphabet	A, B, C, D, E, F, G, H, I, J
Numbers	1, 2, 3, 4, 5, 6, 7, 8, 9, 10

■ ■ Prerequisites

Install the following software before starting. All are free and open source.

Software	Version	Download URL	Purpose
Python	3.11+	python.org/downloads	Backend + ML
Node.js	18 LTS+	nodejs.org	Frontend
Git	Latest	git-scm.com	Clone repo
Webcam	Any	Built-in or USB	Data collection

1. Go to python.org/downloads and download Python 3.11
2. During install — CHECK ■ 'Add Python to PATH'
3. Verify in terminal:

```
python --version  
  
# Expected: Python 3.11.x
```

1. Go to nodejs.org and download the LTS version
2. Install normally with default settings
3. Verify:

```
node --version    # Expected: v18.x.x or v20.x.x  
  
npm --version     # Expected: 9.x.x or 10.x.x
```

1. Go to git-scm.com and install with default settings
2. Verify:

```
git --version
```

■ Step 4 — Clone the Project

```
git clone https://github.com/himanshujagdishpatil914/Himanshu.git
cd Himanshu
```

Your folder structure after cloning:

```
Himanshu/
├── backend/
│   ├── app.py
│   ├── requirements.txt
│   └── model/
│       ├── collect_data.py
│       └── train.py
├── frontend/
│   ├── package.json
│   └── src/
```

■ Step 5 — Setup Python Backend

5.1 Navigate to backend folder

```
cd backend
```

5.2 Create a Python Virtual Environment

```
python -m venv venv

# Activate – Windows:
venv\Scripts\activate

# Activate – Mac / Linux:
source venv/bin/activate
```

■ You will see (venv) at the start of your terminal line when activated.

5.3 Install Python Packages

```
pip install -r requirements.txt
```

■ This takes 5–10 minutes. TensorFlow and MediaPipe are large packages.

5.4 Verify Installation

```
python -c "import mediapipe; import tensorflow; print('All good!')"

# Expected output: All good!
```

■ Step 6 — Collect Training Data

■ You need a working webcam for this step.

```
# Make sure you are in backend/ with venv activated  
cd model  
python collect_data.py
```

Key	Action
ENTER	Record one sample (30 frames)
N	Move to next sign
P	Move to previous sign
Q	Quit data collection

Samples / Sign	Expected Accuracy	Time (40 signs)
50	~80%	~10 minutes
100	~90%	~20 minutes
200 (Recommended)	~95–98%	~40 minutes

- Use good lighting — face a window or bright lamp
- Use a plain background — white or light-coloured wall
- Slightly vary your hand angle and distance each sample
- Keep signs clear and deliberate
- Do not collect in dark rooms
- Do not rush — blurry frames reduce accuracy

■ Step 7 — Train the AI Model

```
# Still inside backend/model/ with venv activated  
python train.py
```

```
[INFO] Loading data for 40 signs ...  
hello          → 200 samples  
yes            → 200 samples  
...  
[INFO] Dataset: 8000 samples, shape (8000, 30, 63)  
[INFO] Train: 5780 | Val: 1020 | Test: 1200  
[INFO] Training ...  
Epoch 1/200 - accuracy: 0.43  
Epoch 10/200 - accuracy: 0.78  
Epoch 50/200 - accuracy: 0.96  
...  
[RESULT] Test Accuracy: 96.5% | Loss: 0.1243  
[SAVED] Model -> backend/data/sign_model.h5 ■  
[SAVED] Labels -> backend/data/label_encoder.npy
```

- Training takes 10–30 minutes depending on your hardware.
- A GPU (NVIDIA) will speed up training significantly.

```
backend/data/  
■■■ sign_model.h5      ← The trained AI brain  
■■■ label_encoder.npy  ← Sign label names  
■■■ training_history.json ← Accuracy/loss history
```

■ Step 8 — Start the Backend Server

```
# Go back to backend/ folder

cd ..          # (if you are in backend/model/)

# Make sure venv is still active, then:

uvicorn app:app --host 0.0.0.0 --port 8000 --reload
```

```
INFO: Started server process
INFO: Waiting for application startup.
INFO: Model loaded | 40 classes ■
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000
```

Open your browser and visit the following URL to confirm the server is running:

```
http://localhost:8000/api/health
```

You should see:

```
{
  "status":      "ok",
  "model_loaded": true,
  "sign_count":  40
}
```

■ Keep this terminal open — do not close it while using the app!

Method	Endpoint	Description
WS	/ws/predict	Real-time sign prediction via WebSocket
POST	/api/tts	Convert text to speech (returns MP3)
GET	/api/signs	List all supported signs
GET	/api/health	Server health check

■■ Step 9 — Setup and Start Frontend

Open a NEW terminal window (keep the backend terminal running!).

9.1 Go to frontend folder

```
cd Himanshu/frontend
```

9.2 Install Node packages

```
npm install
```

■ Takes 2–5 minutes.

9.3 Start the app

```
npm start
```

```
Compiled successfully!
```

```
You can now view sign-language-translator in the browser.
```

```
Local:    http://localhost:3000
```

```
Network:  http://192.168.x.x:3000
```

■ The browser will automatically open <http://localhost:3000>

■ Step 10 — Using the App

1	Click ■ Start Detecting button
2	Allow camera access when the browser asks
3	Show your hand sign clearly to the camera
4	The detected sign appears with a confidence bar
5	Signs auto-add to the Sentence Builder below
6	Click ■ Speak to hear the full sentence
7	Click ■ Undo to remove the last word
8	Click ■ Clear to start a new sentence

Browse all 40 supported signs organised by category.

Use the search box to quickly find a specific sign.

Signs are grouped into: Common Phrases, Expressions, Actions, Alphabet, Numbers.

Mode	Description
■ Words	Displays detected signs as individual coloured chips. Click any chip to remove it.
■ Sentence	Joins all chips into a full readable sentence. Use Speak to hear it.

■ Daily Running Cheatsheet

After the first-time setup, use these two commands every time you want to run the project.

```
cd Himanshu/backend

# Windows:

venv\Scripts\activate

# Mac / Linux:

source venv/bin/activate

uvicorn app:app --port 8000
```

```
cd Himanshu/frontend

npm start
```

Then open your browser at <http://localhost:3000> ■

■ Common Errors and Fixes

Error / Problem	Fix
ModuleNotFoundError	Run: <code>pip install -r requirements.txt</code> inside venv
Camera not found / denied	Click Allow in browser camera permission popup
WebSocket connection failed	Make sure backend is running on port 8000
npm: command not found	Install Node.js from nodejs.org
venv not activated	Run: <code>venv\Scripts\activate</code> (Windows) or <code>source venv/bin/activate</code>
Model not loaded (demo mode)	Run <code>train.py</code> first to generate <code>sign_model.h5</code>
Port 8000 already in use	Use: <code>uvicorn app:app --port 8001</code> and update frontend <code>.env</code>
Low accuracy (<80%)	Collect more samples (200/sign) with better lighting
App shows blank page	Check browser console for errors, ensure npm start ran successfully

■ Minimum System Requirements

Component	Minimum	Recommended
RAM	8 GB	16 GB
CPU	Intel i5 / Ryzen 5	Intel i7 / Ryzen 7
GPU	Not required	NVIDIA (speeds up training)
Storage	5 GB free	10 GB free
Webcam	Any webcam	HD 1080p
OS	Windows 10 / Ubuntu 20 / macOS 11	Windows 11 / Ubuntu 22 / macOS 13

■ Improving Accuracy

Tip	Impact
Collect 200+ samples per sign	■■■ High
Vary lighting conditions	■■ Medium
Use plain background	■■ Medium
Vary hand size and angle	■■ Medium
Run more training epochs (200+)	■ Low–Medium
Use GPU for training	■ Speed only
Add data augmentation (flip/rotate)	■■ Medium

■ Recommended Dataset (Kaggle)

To boost accuracy further, use a pre-built dataset alongside your own collected data.

Dataset	Signs	Format	Link
Google ASL Signs (Kaggle)	250	MediaPipe landmarks	kaggle.com/competitions/asl-signs
Sign Language MNIST	24	CSV pixel images	kaggle.com/datasets/datamunge/sign-language-mnist
ASL Alphabet (Kaggle)	29	JPG images	kaggle.com/datasets/grassknoted/asl-alphabet

Download Command:

```
pip install kaggle

# Place kaggle.json in ~/.kaggle/

# Best option – pre-extracted MediaPipe landmarks:
kaggle competitions download -c asl-signs -p ./backend/data/

# Easy starter:
kaggle datasets download -d datamunge/sign-language-mnist -p ./backend/data/
```

■ You're All Set!



■ himanshujagdishpatil914@gmail.com

■ github.com/himanshujagdishpatil914/Himanshu